

Zeitreihenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Maik Poetzsch
	Sebastian Seipel	

2026-09-17

Inhaltsverzeichnis

Preamble	4
Intro	5
1 1 Einordnung: Zeitreihen als Messprozess	6
2 1 Einordnung: Zeitreihen als Messprozess	7
3 2 Vorverarbeitung & Validierung einer Zeitreihe	12
4 2 Vorverarbeitung & Validierung einer Zeitreihe	13
5 3 Resampling & Alignment	18
6 3 Resampling & Alignment	19
7 4 Glättung & Rauschreduktion	25
8 4 Glättung & Rauschreduktion	26
9 5 Trend & Saison	34
10 5 Trend & Saison	35
11 6 Lags & Autokorrelation	42
12 6 Lags & Autokorrelation	43
13 7 Anomalien	49
14 7 Anomalien	50
15 8 Forecasting-Baselines & Bewertung	57
16 8 Forecasting-Baselines & Bewertung	58
17 9 Lernzielkontrolle	65

18 9 Lernzielkontrolle	66
19 10 Übung: Zeitreihen-Workflow	68
20 10 Übung: Zeitreihen-Workflow	69
21 11 Klausurfragen	71
22 11 Klausurfragen	72

Preamble



Bausteine Computergestützter Datenanalyse. “Zeitreihenanalyse” von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Maik Poetzsch und Sebastian Seipel ist lizenziert unter [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Das Werk ist abrufbar unter <https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten>. Ausgenommen von der Lizenz sind alle Logos und anders gekennzeichneten Inhalte. 2026

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Maik Poetzsch, und Sebastian Seipel. 2024. „Bausteine Computergestützter Datenanalyse. Methodenbaustein Zeitreihenanalyse“. <https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten>.

BibTeX-Vorlage

```
@misc{BCD-Styleguide-2024,  
  title={Bausteine Computergestützter Datenanalyse. Methodenbaustein Zeitreihenanalyse},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Poetzsch, Maik},  
  year={2024},  
  url={https://github.com/bausteine-der-datenanalyse/m-analyse-von-zeitdaten}}
```

Intro

Voraussetzungen

- Python-Grundlagen
- NumPy (Arrays, grundlegende Operationen)
- Matplotlib (Visualisierung)
- Grundverständnis von Pandas (Series, DataFrame)

Verwendete Pakete und Datensätze

Pakete

- [NumPy](#)
- [Matplotlib](#)
- [Pandas](#)

Datensätze

Bearbeitungszeit

Geschätzte Bearbeitungszeit: 2h

Lernziele

- Zeitreihen strukturiert vorverarbeiten (Zeitindex, Missing, Ausreißer)
- Resampling und Alignment methodisch korrekt einsetzen
- Glättung so wählen, dass sie zur Fragestellung passt
- Trend und Saisonanteile erkennen und interpretieren
- zeitliche Abhängigkeiten mit Lag/Autokorrelation diagnostizieren
- Anomalien mit einfachen, begründbaren Regeln detektieren
- Forecast-Baselines implementieren und mit Fehlermaßen bewerten

1 1 Einordnung: Zeitreihen als Messprozess

2 1 Einordnung: Zeitreihen als Messprozess

Eine Zeitreihe ist das Ergebnis eines **Messprozesses über die Zeit**.

Typische Beispiele:

- Sensordaten
- Energieverbrauch
- Web-Traffic
- Finanzdaten
- Maschinendaten in Produktionsprozessen

Wichtig ist nicht nur der Zahlenwert selbst, sondern:

- **Abtastrate** (stündlich, minütlich, unregelmäßig?)
- **Messqualität**
- **Strukturelle Muster** (Trend, Periodik)
- **Störungen** (Rauschen, Ausreißer)

Merke

Zeitreihenanalyse beginnt nicht mit einem Modell, sondern mit dem Verständnis des Messprozesses.

2.1 1.1 Arbeitsdatensatz erzeugen

Wir erstellen eine synthetische Zeitreihe mit:

- Trend
- Tagesperiodik
- Rauschen
- Ausreißern

Diese Struktur ist typisch für reale technische Messdaten.

2.2 Pandas (empfohlen)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

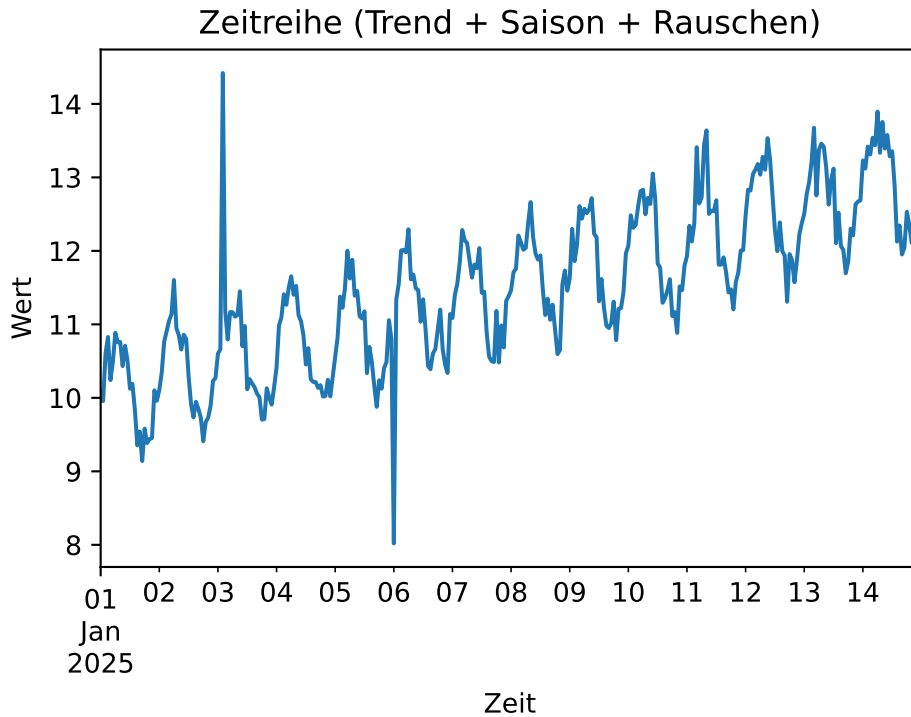
# Zeitindex: 14 Tage, stündliche Messung
t = pd.date_range("2025-01-01", periods=24*14, freq="h")

trend = np.linspace(0, 3, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.25 * rng.normal(size=len(t))

y = 10 + trend + season + noise
y = pd.Series(y, index=t, name="signal")

# künstliche Ausreißer
y.iloc[50] += 3.5
y.iloc[120] -= 2.8

y.plot()
plt.title("Zeitreihe (Trend + Saison + Rauschen)")
plt.xlabel("Zeit")
plt.ylabel("Wert")
plt.show()
```



2.3 NumPy (Prinzip)

Hier modellieren wir Zeit als einfachen Sample-Index.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

n = 24*14
x = np.arange(n)

trend = np.linspace(0, 3, n)
season = 0.8 * np.sin(2*np.pi*(x % 24)/24)
noise = 0.25 * rng.normal(size=n)

y = 10 + trend + season + noise

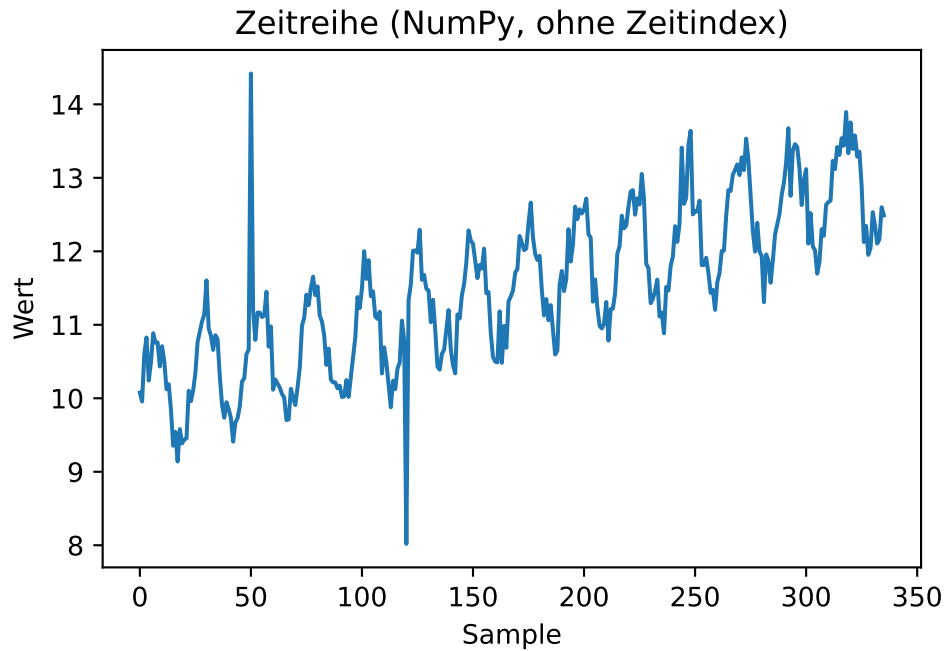
# Ausreißer
```

```

y[50] += 3.5
y[120] -= 2.8

plt.plot(y)
plt.title("Zeitreihe (NumPy, ohne Zeitindex)")
plt.xlabel("Sample")
plt.ylabel("Wert")
plt.show()

```



2.4 1.2 Typische Analysefragen

Bei Zeitreihen treten immer wieder ähnliche Fragestellungen auf:

- Gibt es einen **Trend**?
- Existiert eine **Periodik** (z. B. Tages- oder Wochenmuster)?
- Wie stark ist das **Rauschen**?
- Gibt es **Anomalien**?
- Wie kann ich zukünftige Werte grob abschätzen?

Methodischer Blick

Bevor Sie glätten oder modellieren:

1. Visualisieren
2. Zeitskala prüfen
3. Struktur erkennen
4. Erst dann weiter analysieren

2.5 1.3 Strukturmodell einer Zeitreihe

Eine Zeitreihe lässt sich häufig konzeptionell darstellen als:

Signal = Trend + Saison + Rauschen + Ausreißer

Diese Zerlegung ist kein physikalisches Gesetz, sondern ein **Denkmodell**, das hilft, Muster systematisch zu analysieren.

Achtung

Nicht jedes wiederkehrende Muster ist automatisch „Saison“.
Regelmäßigkeit muss zur Abtastrate und zum Prozess plausibel passen.

2.6 1.4 Mini-Reflexion

Beantworten Sie:

1. Welche Periodik wurde hier modelliert?
2. Welche Rolle spielt die Abtastrate?
3. Würde eine tägliche Aggregation den Saisonanteil sichtbar oder unsichtbar machen?
4. Wie würden sich Ausreißer bei starker Glättung verhalten?

Diese Fragen begleiten uns durch den gesamten Methodenbaustein.

3 2 Vorverarbeitung & Validierung einer Zeitreihe

4 2 Vorverarbeitung & Validierung einer Zeitreihe

Bevor eine Zeitreihe analysiert wird, muss sie **formal korrekt und konsistent** sein.

Viele Fehler in späteren Analysen entstehen durch:

- unsortierte Zeitstempel
- doppelte Zeitpunkte
- fehlende Werte
- unregelmäßige Abtastung
- implizite Zeitlücken

Merke

Zeitreihenanalyse ist nur so gut wie die Datenbasis. Validierung ist kein optionaler Schritt.

4.1 2.1 Beispiel-Zeitreihe mit Problemen

Wir erzeugen bewusst eine fehlerhafte Zeitreihe (unsortiert, Duplikate, Missing Values).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(1)

t = pd.date_range("2025-01-01", periods=24*3, freq="h")
y = pd.Series(10 + rng.normal(size=len(t)), index=t, name="signal")

# Probleme erzeugen
y = y.sample(frac=1)                    # unsortiert
y = pd.concat([y, y.iloc[5:8]])         # Duplikate (append ist deprecated/entfernt)
y.iloc[10:15] = np.nan                 # fehlende Werte

y.head()
```

```
2025-01-02 17:00:00    9.316773
2025-01-03 15:00:00    8.892627
2025-01-01 03:00:00    8.696843
2025-01-02 12:00:00    9.485994
2025-01-02 13:00:00    8.351925
Name: signal, dtype: float64
```

4.2 2.2 Schritt 1: Sortierung sicherstellen

```
y = y.sort_index()
```

4.3 2.3 Schritt 2: Duplikate erkennen und behandeln

Duplikate erkennen:

```
dup_mask = y.index.duplicated(keep=False)
y[dup_mask].head(10)
```

```
2025-01-01 19:00:00    9.742808
2025-01-01 19:00:00    9.742808
2025-01-03 06:00:00   10.879161
2025-01-03 06:00:00   10.879161
2025-01-03 19:00:00   10.759520
2025-01-03 19:00:00   10.759520
Name: signal, dtype: float64
```

Duplikate müssen methodisch entschieden behandelt werden, z. B.:

- Mittelwert pro Zeitstempel (typisch bei Messwerten)
- ersten/letzten Wert behalten (typisch bei Status-Updates)
- entfernen (wenn klar fehlerhaft)

Beispiel: Mittelwert pro Zeitstempel

```
y = y.groupby(level=0).mean()
```

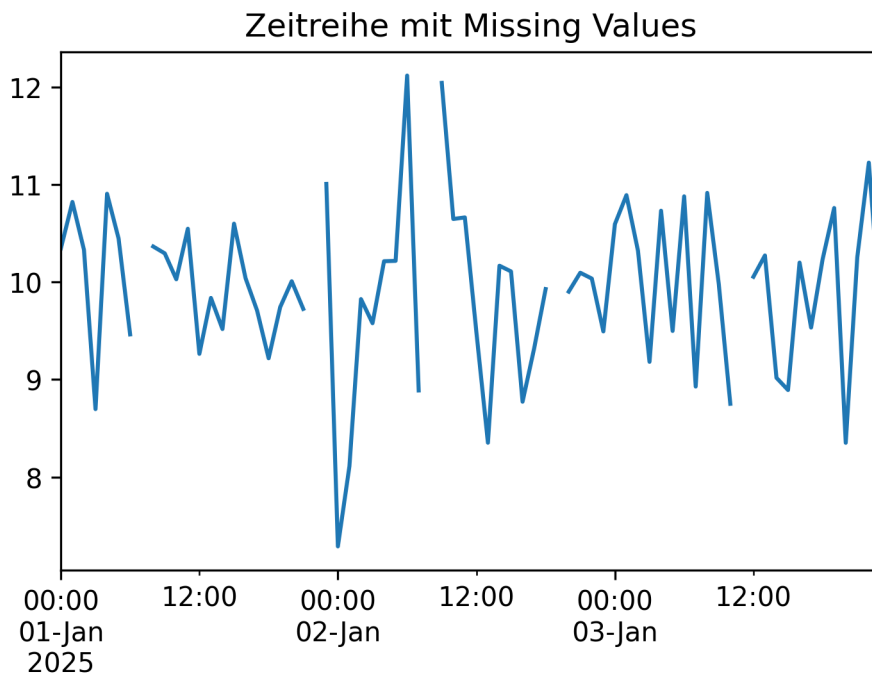
4.4 2.4 Schritt 3: Fehlende Werte analysieren

```
y.isna().sum()
```

```
np.int64(5)
```

4.4.1 Visualisierung der Lücken

```
import matplotlib.pyplot as plt  
  
y.plot(title="Zeitreihe mit Missing Values")  
plt.show()
```



4.4.2 Behandlungsmöglichkeiten

```
y_ffill = y.ffmpeg()
y_interp = y.interpolate("time")
```

Entscheidungshilfe

- `ffill`: sinnvoll bei Zustandsdaten („gilt bis zur nächsten Messung“)
- `interpolate("time")`: sinnvoll bei kontinuierlichen Messgrößen

4.5 2.5 Schritt 4: Abtastrate prüfen

Ist die Zeitreihe regelmäßig?

```
diffs = y.index.to_series().diff()
diffs.value_counts().head()
```

```
0 days 01:00:00    71
Name: count, dtype: int64
```

Wenn mehrere Zeitabstände auftreten → unregelmäßige Abtastung oder Zeitlücken.

4.6 2.6 Zeitlücken explizit machen

Mit `asfreq` wird die erwartete Frequenz erzwungen. Fehlende Zeitpunkte werden als NaN sichtbar.

```
y_regular = y.asfreq("h")
y_regular.isna().sum()
```

```
np.int64(5)
```

Achtung

Ohne explizite Frequenz können Zeitlücken unsichtbar bleiben.

4.7 2.7 Validierungs-Checkliste

Bevor Sie weiter analysieren:

- ☐ Index ist `DatetimeIndex`
- ☐ Zeitreihe ist sortiert
- ☐ Duplikate identifiziert und begründet behandelt
- ☐ Fehlende Werte identifiziert und Strategie gewählt
- ☐ Abtastrate geprüft
- ☐ Frequenz (falls sinnvoll) explizit gesetzt

4.8 2.8 Mini-Aufgabe

Erzeugen Sie selbst eine Zeitreihe mit:

- unregelmäßiger Abtastung
- 5 fehlenden Zeitpunkten
- 3 Duplikaten

Implementieren Sie anschließend eine saubere Validierung gemäß Checkliste.

5 3 Resampling & Alignment

6 3 Resampling & Alignment

Zeitreihen liegen häufig in unterschiedlichen Abtastraten vor oder müssen auf eine andere Zeitskala transformiert werden.

Typische Gründe:

- Vergleich zweier Zeitreihen
- Reduktion von Rauschen durch Aggregation
- Anpassung an Reporting-Intervalle
- Vorbereitung für Modellierung

i Merke

Resampling bedeutet: Änderung der Zeitskala. Dabei muss entschieden werden zwischen Aggregation und Interpolation.

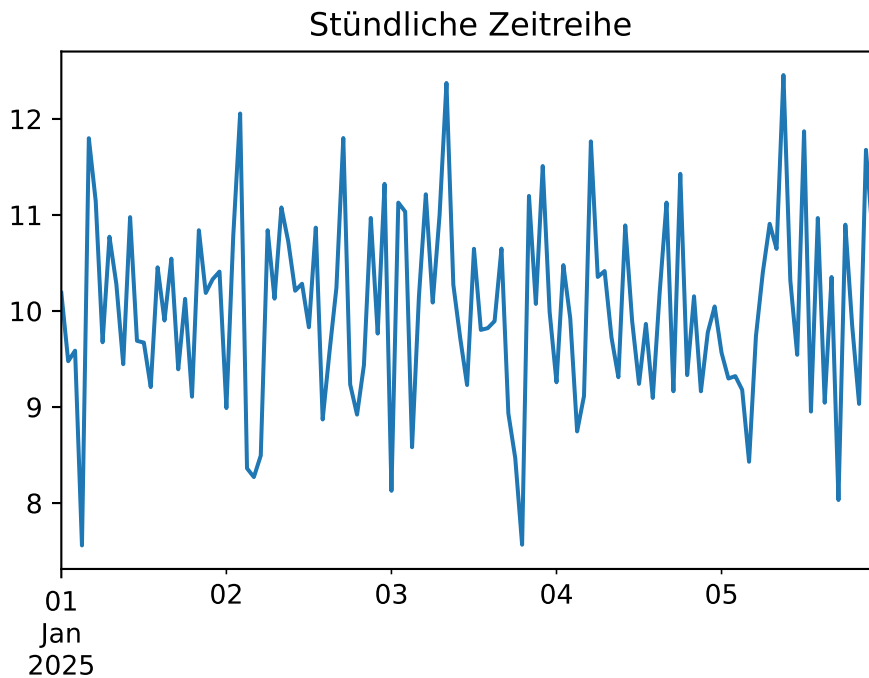
6.1 3.1 Ausgangszeitreihe (stündlich)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(2)

t = pd.date_range("2025-01-01", periods=24*5, freq="h")
y = pd.Series(10 + rng.normal(size=len(t)), index=t, name="signal")

y.plot(title="Stündliche Zeitreihe")
plt.show()
```

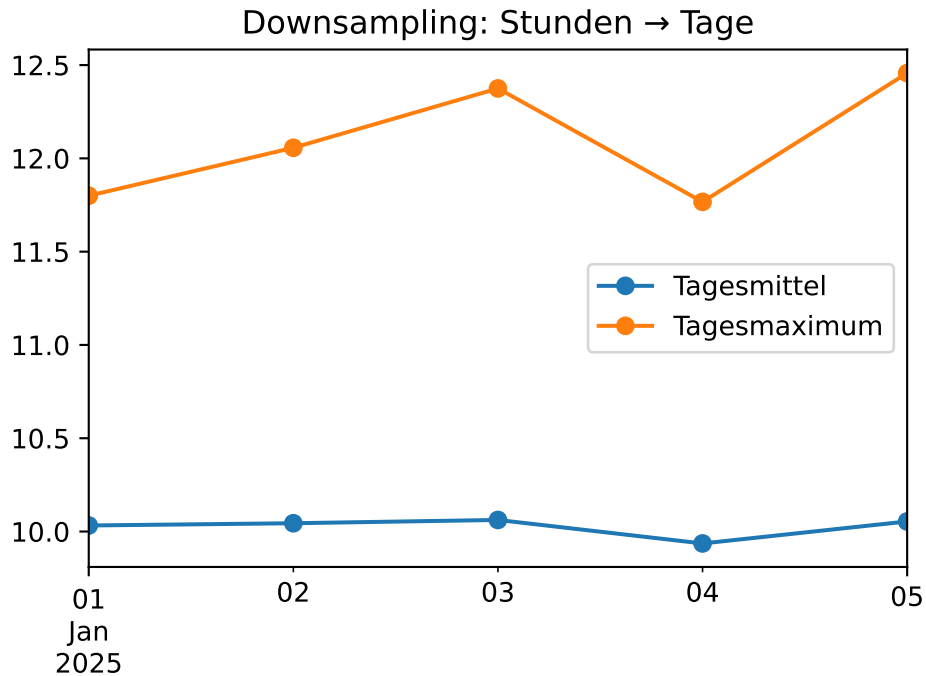


6.2 3.2 Downsampling (Aggregation)

Wir aggregieren von Stunden auf Tage.

```
daily_mean = y.resample("D").mean()
daily_max = y.resample("D").max()

ax = daily_mean.plot(marker="o", label="Tagesmittel")
daily_max.plot(ax=ax, marker="o", label="Tagesmaximum")
plt.legend()
plt.title("Downsampling: Stunden → Tage")
plt.show()
```



💡 Entscheidungshilfe

- Mittelwert → glättet Schwankungen
- Maximum → zeigt Extremwerte
- Summe → sinnvoll bei Mengen (z. B. Energieverbrauch)

6.3 3.3 Upsampling (Interpolation)

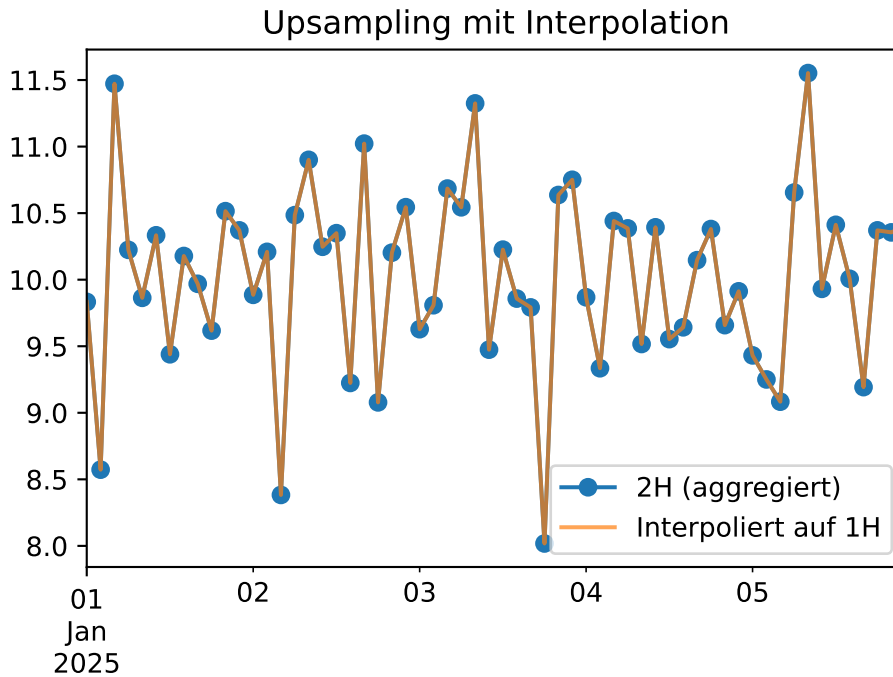
Wir erhöhen die zeitliche Auflösung.

```
y_2h = y.resample("2h").mean()  
y_up = y_2h.resample("h").interpolate("time")
```

```

ax = y_2h.plot(marker="o", label="2H (aggregiert)")
y_up.plot(ax=ax, alpha=0.7, label="Interpoliert auf 1H")
plt.legend()
plt.title("Upsampling mit Interpolation")
plt.show()

```



⚠ Achtung

Interpolation erzeugt keine neuen Messinformationen, sondern modelliert einen plausiblen Verlauf.

6.4 3.4 Alignment zweier Zeitreihen

Zwei Zeitreihen mit unterschiedlicher Abtastrate:

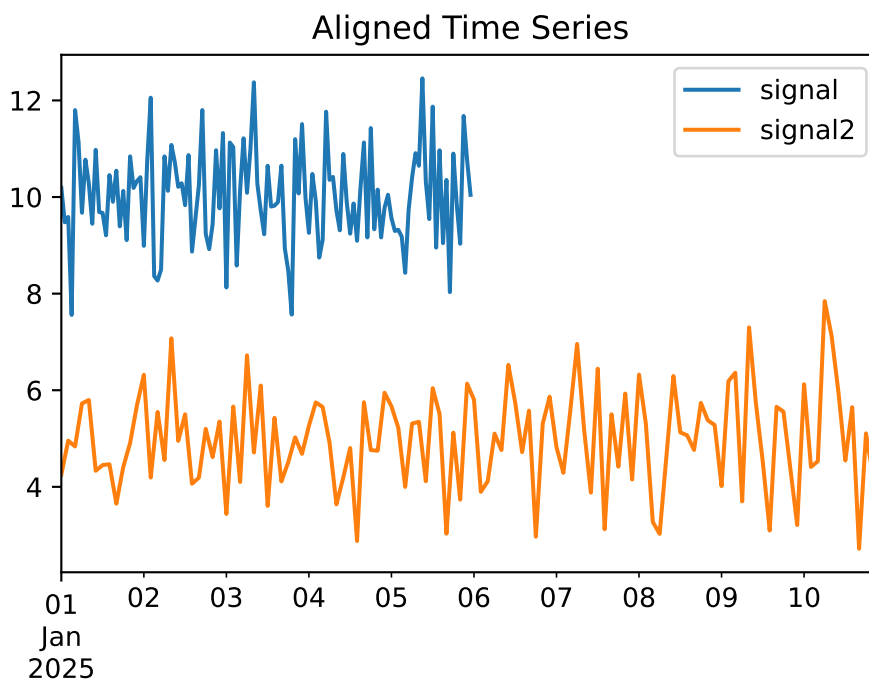
```
t2 = pd.date_range("2025-01-01", periods=24*5, freq="2h")
y2 = pd.Series(5 + rng.normal(size=len(t2)), index=t2, name="signal2")
```

Zur gemeinsamen Analyse müssen beide Reihen dieselbe Frequenz haben.

```
y2_aligned = y2.resample("h").interpolate("time")

df = pd.concat([y, y2_aligned], axis=1)

df.plot(title="Aligned Time Series")
plt.show()
```



i Merke

Alignment bedeutet: - gleiche Frequenz - gleicher Zeitbereich - gleiche Zeitzone

6.5 3.5 Häufige Fehlerquellen

- Vergleiche ohne vorheriges Resampling
 - Mischung von lokaler Zeit und UTC
 - Aggregation ohne Berücksichtigung der Datentypen (z. B. Summe vs Mittelwert)
-

6.6 3.6 Mini-Aufgabe

1. Erzeugen Sie eine minütliche Zeitreihe über 2 Tage.
2. Aggregieren Sie auf Stundenmittel.
3. Vergleichen Sie visuell:
 - Original
 - Stundenmittel
4. Interpretieren Sie den Informationsverlust.

7 4 Glättung & Rauschreduktion

8 4 Glättung & Rauschreduktion

Glättung ist eine der wichtigsten Standardmethoden in der Zeitreihenanalyse. Sie hilft dabei,

- Rauschen zu reduzieren,
- Trend/Saison besser sichtbar zu machen,
- eine **Baseline** zu definieren (z. B. für Anomalien).

i Merke

Die Fensterbreite ist eine methodische Entscheidung: Sie sollte zur **Zeitskala der Fragestellung** passen.

8.1 4.1 Beispielsignal

Wir nutzen eine Zeitreihe mit Tagesmuster, Rauschen und einzelnen Ausreißern.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

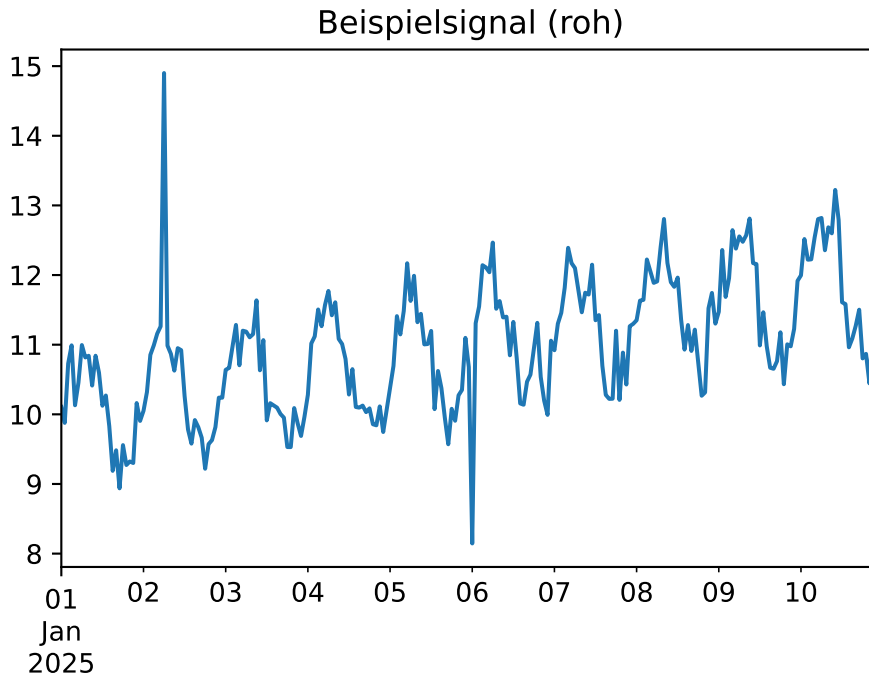
rng = np.random.default_rng(42)

t = pd.date_range("2025-01-01", periods=24*10, freq="h") # 10 Tage stündlich
trend = np.linspace(0, 2, len(t))
season = 0.9 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.35 * rng.normal(size=len(t))

s = pd.Series(10 + trend + season + noise, index=t, name="signal")

# Ausreißer
s.iloc[30] += 3.0
s.iloc[120] -= 2.5

s.plot(title="Beispielsignal (roh)")
plt.show()
```



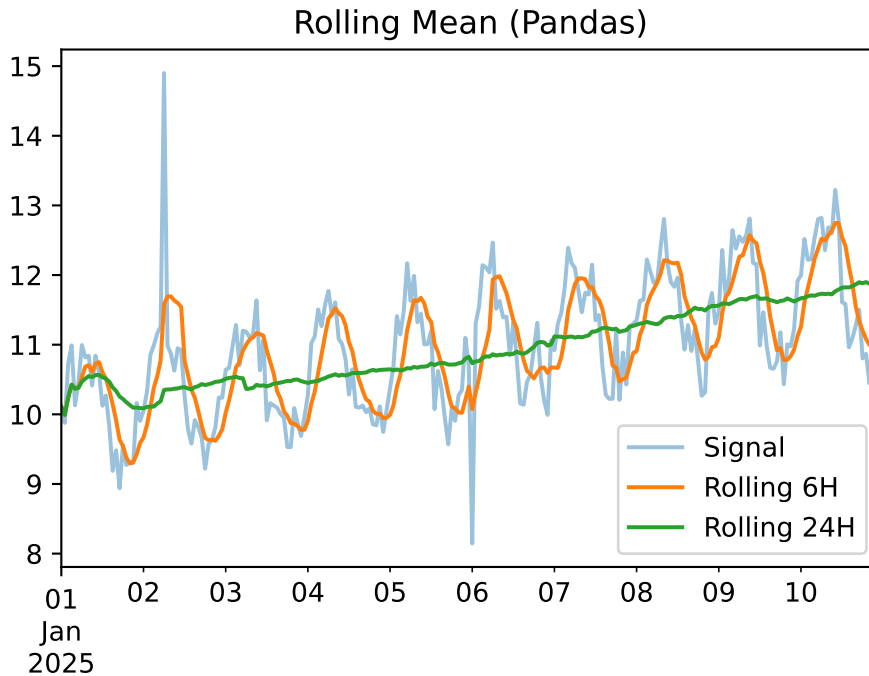
8.2 4.2 Rolling Mean (gleitender Mittelwert)

Rolling Mean ist die klassische Glättung: Mittelwert über ein Fenster.

8.3 Pandas (empfohlen)

```
roll_6h = s.rolling("6h").mean()
roll_24h = s.rolling("24h").mean()

ax = s.plot(alpha=0.45, label="Signal")
roll_6h.plot(ax=ax, label="Rolling 6H")
roll_24h.plot(ax=ax, label="Rolling 24H")
plt.legend()
plt.title("Rolling Mean (Pandas)")
plt.show()
```



💡 Entscheidungshilfe

- **6H:** reduziert kurzfristiges Rauschen, Ereignisse bleiben sichtbar
- **24H:** zeigt eher Grundniveau/Trend, dämpft Tagesmuster stark

8.4 NumPy (Prinzip)

NumPy-Variante: Moving Average als Faltung (`convolve`).

```
import numpy as np
import matplotlib.pyplot as plt

y = s.to_numpy()

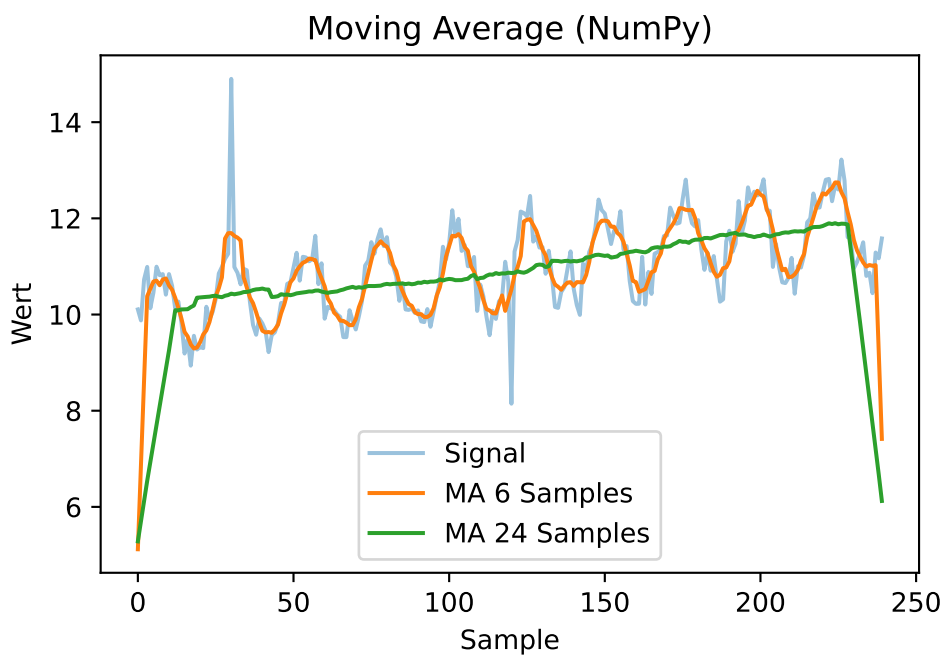
def moving_average(y, window):
    kernel = np.ones(window) / window
    return np.convolve(y, kernel, mode="same")
```

```

ma_6 = moving_average(y, 6)
ma_24 = moving_average(y, 24)

plt.plot(y, alpha=0.45, label="Signal")
plt.plot(ma_6, label="MA 6 Samples")
plt.plot(ma_24, label="MA 24 Samples")
plt.legend()
plt.title("Moving Average (NumPy)")
plt.xlabel("Sample")
plt.ylabel("Wert")
plt.show()

```



⚠ Achtung

Bei der Faltung müssen Sie Randbehandlung bewusst akzeptieren (hier: `mode="same"`). Pandas behandelt Randbereiche konsistent mit `min_periods` und NaNs.

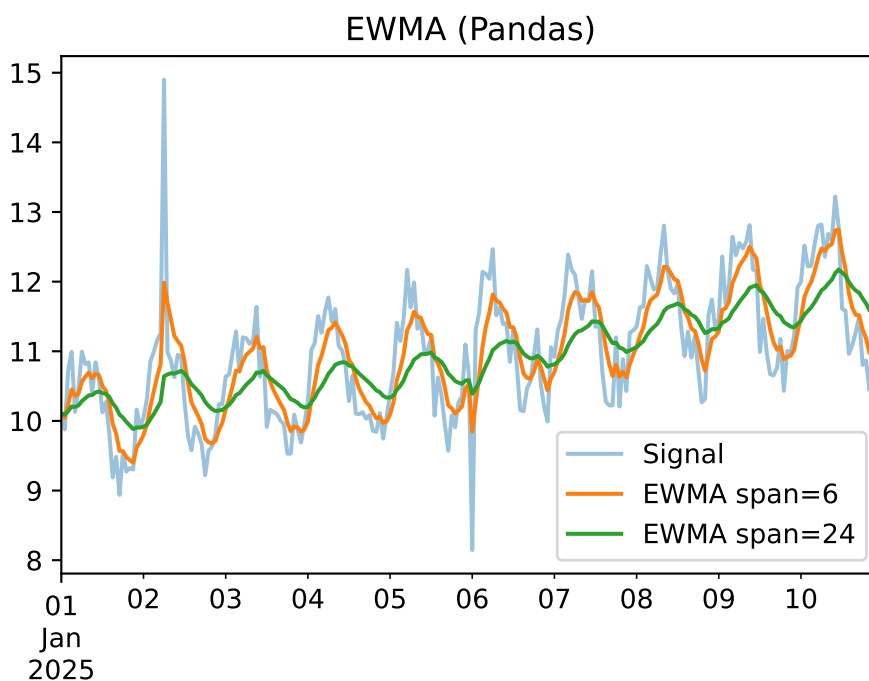
8.5 4.3 EWMA (Exponentially Weighted Moving Average)

EWMA glättet, gewichtet aber **neuere Werte stärker**.

8.6 Pandas (empfohlen)

```
ewm_fast = s.ewm(span=6, adjust=False).mean()
ewm_slow = s.ewm(span=24, adjust=False).mean()

ax = s.plot(alpha=0.45, label="Signal")
ewm_fast.plot(ax=ax, label="EWMA span=6")
ewm_slow.plot(ax=ax, label="EWMA span=24")
plt.legend()
plt.title("EWMA (Pandas)")
plt.show()
```



i Merke

EWMA reagiert schneller auf Änderungen als Rolling Mean, weil ältere Werte exponentiell weniger Einfluss haben.

8.7 NumPy (Prinzip)

EWMA kann rekursiv berechnet werden:

$$m_t = \alpha \cdot y_t + (1 - \alpha) \cdot m_{t-1}$$

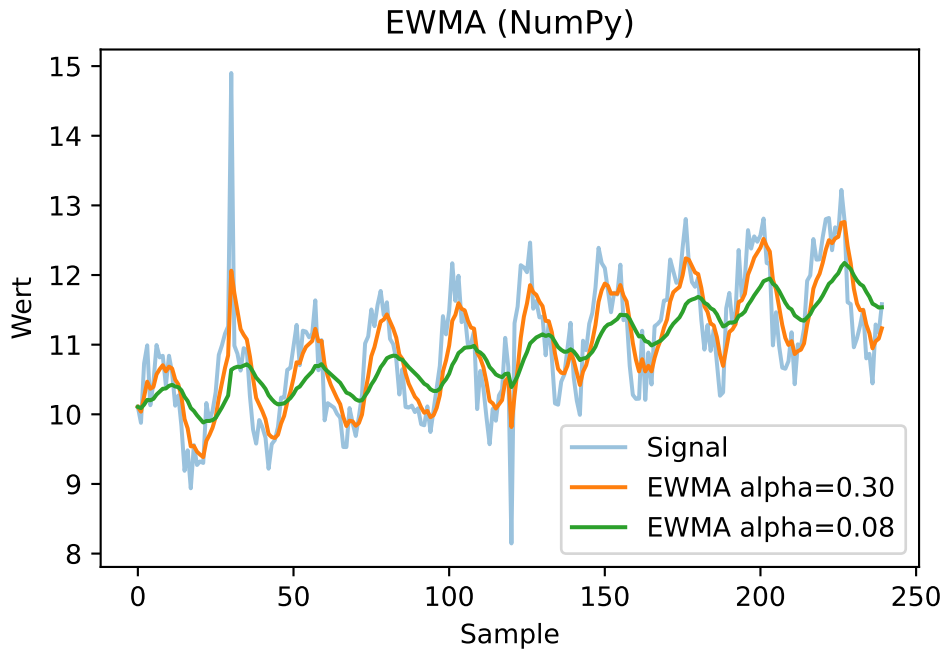
```
import numpy as np
import matplotlib.pyplot as plt

y = s.to_numpy()

def ewma(y, alpha):
    m = np.empty_like(y, dtype=float)
    m[0] = y[0]
    for i in range(1, len(y)):
        m[i] = alpha * y[i] + (1 - alpha) * m[i-1]
    return m

m_fast = ewma(y, alpha=0.30)
m_slow = ewma(y, alpha=0.08)

plt.plot(y, alpha=0.45, label="Signal")
plt.plot(m_fast, label="EWMA alpha=0.30")
plt.plot(m_slow, label="EWMA alpha=0.08")
plt.legend()
plt.title("EWMA (NumPy)")
plt.xlabel("Sample")
plt.ylabel("Wert")
plt.show()
```



8.8 4.4 Methodische Wahl: welches Verfahren wann?

💡 Entscheidungshilfe

Rolling Mean eignet sich gut, wenn...

- Sie ein klar definiertes Zeitfenster mitteln wollen (z. B. „letzte 24h“).

EWMA eignet sich gut, wenn...

- Sie eine smoothie Baseline wollen, die schneller auf Änderungen reagiert.
- Sie Anomalien als Abweichung von einer dynamischen Baseline erkennen möchten.

8.9 4.5 Mini-Aufgaben

1. **Fensterwahl:** Testen Sie Rolling-Fenster von 3H, 12H, 48H.
 - Welche Strukturen werden jeweils sichtbar bzw. verschwinden?
2. **Anomalien & Glättung:** Erhöhen Sie die Ausreißer-Amplitude.
 - Ab welcher Glättungsstärke „verschwindet“ der Ausreißer im geglätteten Signal?
3. **EWMA-Empfindlichkeit:** Variieren Sie `span` (Pandas) bzw. `alpha` (NumPy).

- Welche Parameter reagieren „schnell“, welche „träge“? Begründen Sie.

9 5 Trend & Saison

10 5 Trend & Saison

Viele Zeitreihen enthalten systematische Strukturen:

- **Trend:** langfristige Entwicklung (steigend/fallend)
- **Saison:** periodisch wiederkehrendes Muster
- **Residuum:** verbleibende Schwankung

Diese Zerlegung ist ein **Denkmodell**, das hilft, Struktur systematisch zu analysieren.

Merke

Trend beschreibt eine langsame Veränderung, Saison beschreibt eine regelmäßige Wiederholung.

10.1 5.1 Beispielsignal

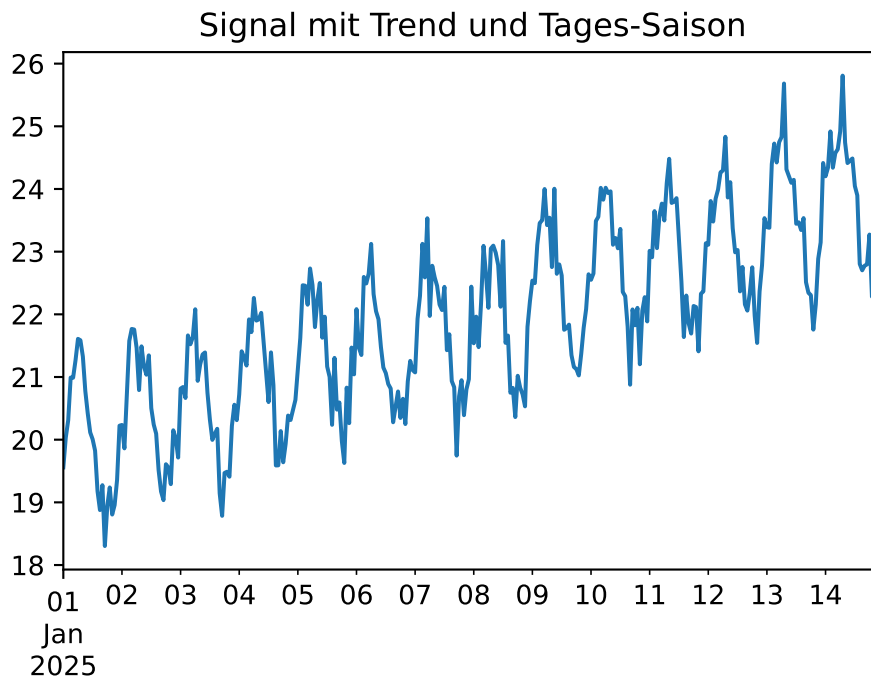
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(10)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 4, len(t))
season = 1.2 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.4 * rng.normal(size=len(t))

s = pd.Series(20 + trend + season + noise, index=t, name="signal")
```

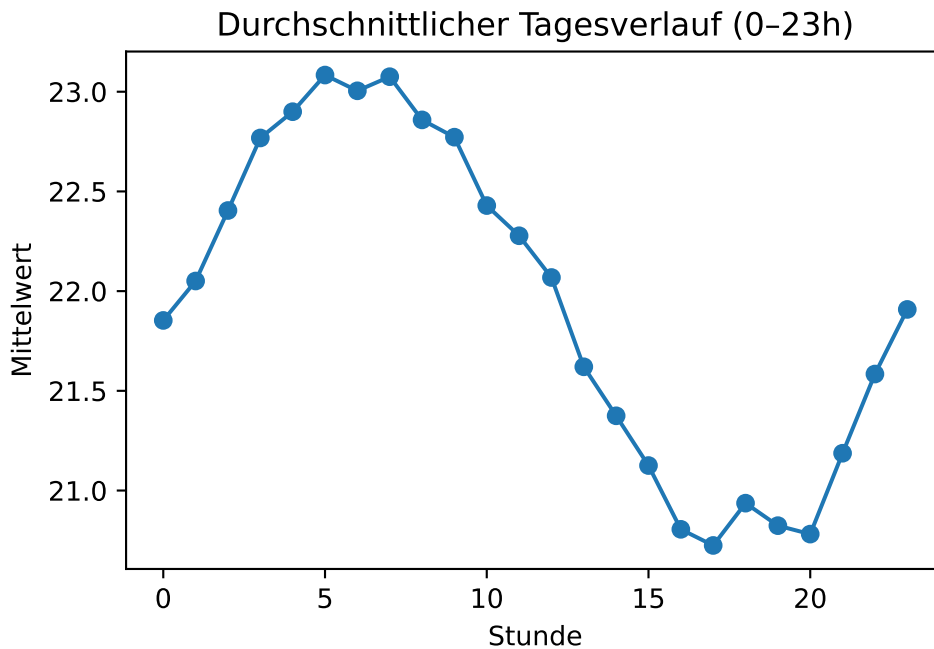
```
s.plot(title="Signal mit Trend und Tages-Saison")  
plt.show()
```



10.2 5.2 Saison sichtbar machen

10.2.1 Mittelwert pro Stunde

```
season_hour = s.groupby(s.index.hour).mean()  
  
season_hour.plot(marker="o")  
plt.title("Durchschnittlicher Tagesverlauf (0-23h)")  
plt.xlabel("Stunde")  
plt.ylabel("Mittelwert")  
plt.show()
```



💡 Interpretation

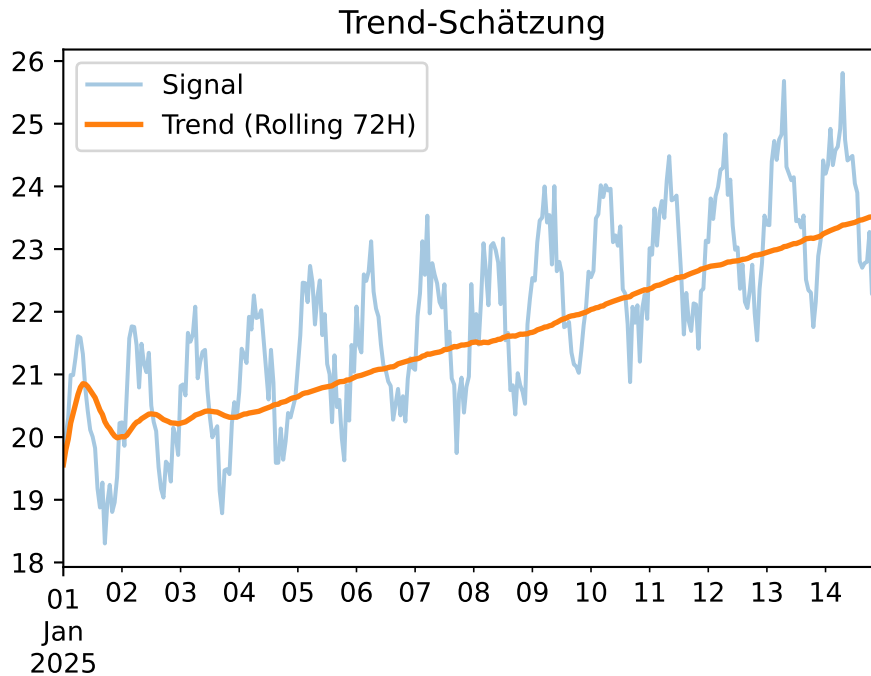
Ein periodisches Muster in dieser Darstellung deutet auf Tages-Saison hin.

10.3 5.3 Trend schätzen (starke Glättung)

10.4 Pandas

```
trend_est = s.rolling("72h").mean()

ax = s.plot(alpha=0.4, label="Signal")
trend_est.plot(ax=ax, linewidth=2, label="Trend (Rolling 72H)")
plt.legend()
plt.title("Trend-Schätzung")
plt.show()
```



10.5 NumPy (Prinzip)

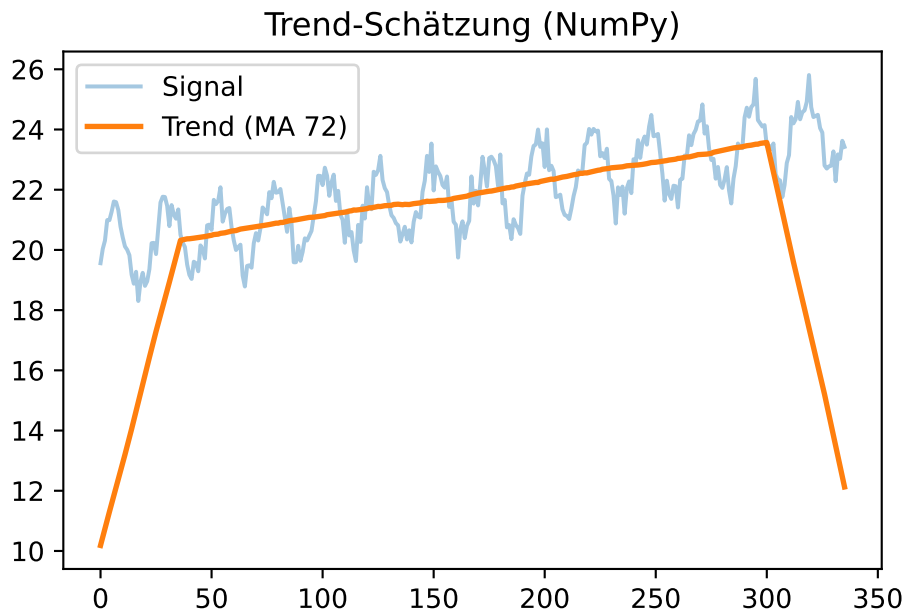
```
import numpy as np

y = s.to_numpy()

def moving_average(y, window):
    kernel = np.ones(window) / window
    return np.convolve(y, kernel, mode="same")

trend_np = moving_average(y, 72)

plt.plot(y, alpha=0.4, label="Signal")
plt.plot(trend_np, linewidth=2, label="Trend (MA 72)")
plt.legend()
plt.title("Trend-Schätzung (NumPy)")
plt.show()
```

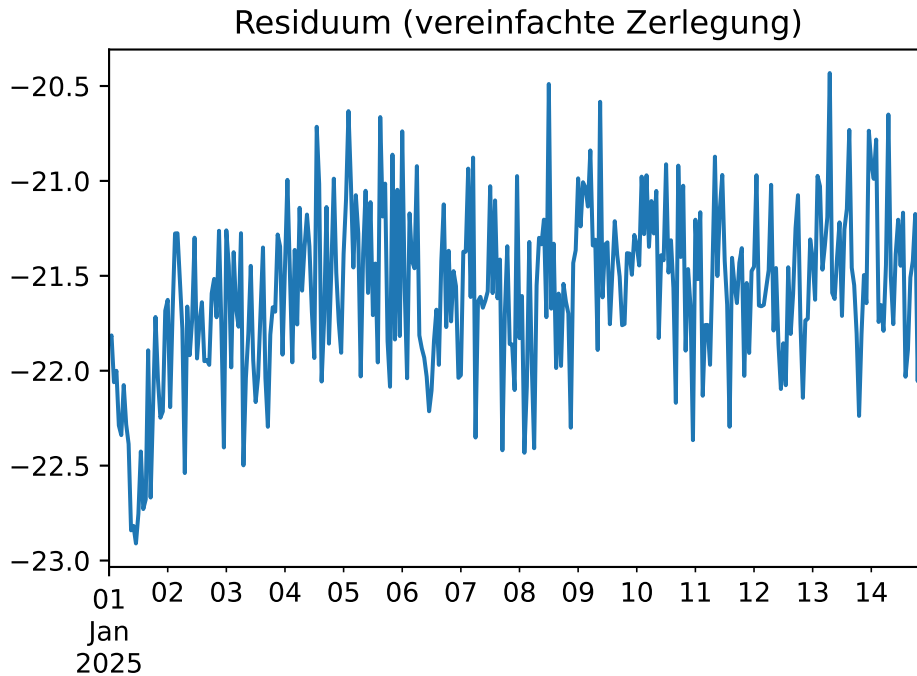


10.6 5.4 Residuum berechnen (vereinfachte Zerlegung)

Wir entfernen Trend und Saison grob:

```
season_map = s.index.hour.map(season_hour)
residual = s - trend_est - season_map

residual.plot(title="Residuum (vereinfachte Zerlegung)")
plt.show()
```



⚠ Achtung

Diese Zerlegung ist didaktisch vereinfacht. In der Praxis existieren robustere Verfahren (z. B. STL). Hier geht es um das Prinzip.

10.7 5.5 Additives vs Multiplikatives Modell

Additives Modell:

Signal = Trend + Saison + Rauschen

Multiplikatives Modell:

Signal = Trend × Saison × Rauschen

Multiplikative Modelle sind sinnvoll, wenn die Amplitude der Saison mit dem Niveau wächst.

10.8 5.6 Mini-Aufgaben

1. Erhöhen Sie die Trendsteigung.
 - Wie beeinflusst das die Erkennung der Saison?
2. Erzeugen Sie eine Wochen-Saison (Periodenlänge 7 Tage).
 - Wie würden Sie diese sichtbar machen?
3. Testen Sie ein sehr großes Rolling-Fenster (z. B. 120H).
 - Welche Struktur bleibt übrig?

11 6 Lags & Autokorrelation

12 6 Lags & Autokorrelation

Zeitreihen unterscheiden sich von klassischen Datensätzen, weil Beobachtungen **nicht unabhängig** sind.

Der Wert zum Zeitpunkt t hängt häufig von vorherigen Werten ab.

Diese zeitliche Abhängigkeit untersuchen wir mit:

- **Lags**
- **Autokorrelation**

i Merke

Zeitliche Abhängigkeit ist eine zentrale Eigenschaft von Zeitreihen. Ohne sie gäbe es keine sinnvolle Prognose.

12.1 6.1 Beispielsignal

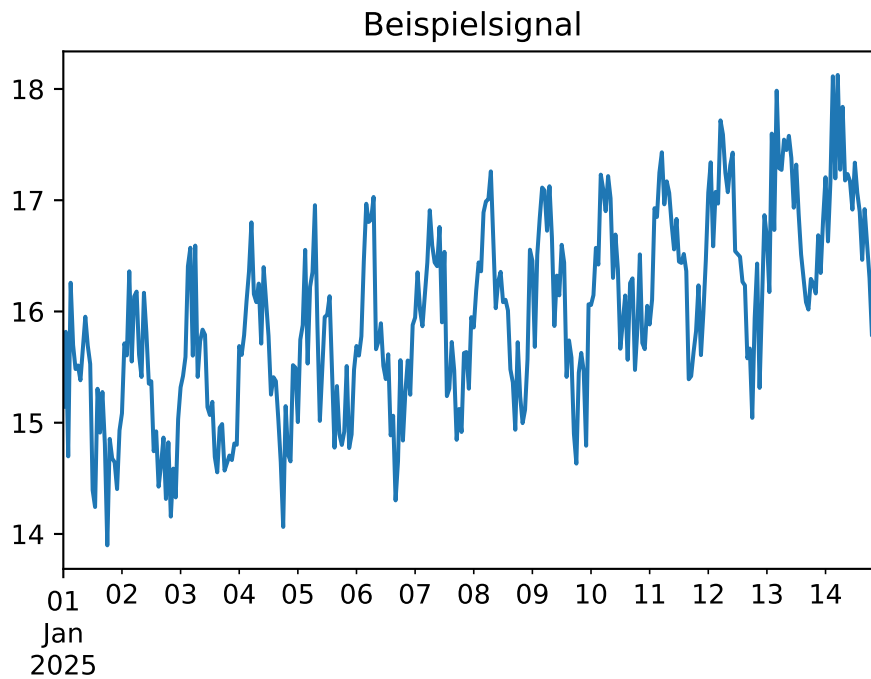
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(21)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.4 * rng.normal(size=len(t))
```

```
s = pd.Series(15 + trend + season + noise, index=t, name="signal")

s.plot(title="Beispielsignal")
plt.show()
```



12.2 6.2 Lag-Features

Ein Lag verschiebt die Zeitreihe nach hinten.

12.3 Pandas

```
lag_1h = s.shift(1)
lag_24h = s.shift(24)

df_lags = pd.DataFrame({
```

```

    "y": s,
    "lag_1h": lag_1h,
    "lag_24h": lag_24h
})

```

```
df_lags.head()
```

	y	lag_1h	lag_24h
2025-01-01 00:00:00	15.143509	NaN	NaN
2025-01-01 01:00:00	15.817296	15.143509	NaN
2025-01-01 02:00:00	14.697408	15.817296	NaN
2025-01-01 03:00:00	16.258241	14.697408	NaN
2025-01-01 04:00:00	15.697774	16.258241	NaN

12.4 NumPy (Prinzip)

```

y = s.to_numpy()

lag1 = np.roll(y, 1)
lag24 = np.roll(y, 24)

lag1[0] = np.nan
lag24[:24] = np.nan

```

Interpretation

Ein hoher Zusammenhang zwischen $y(t)$ und $y(t-24)$ deutet auf eine Tagesperiodik hin.

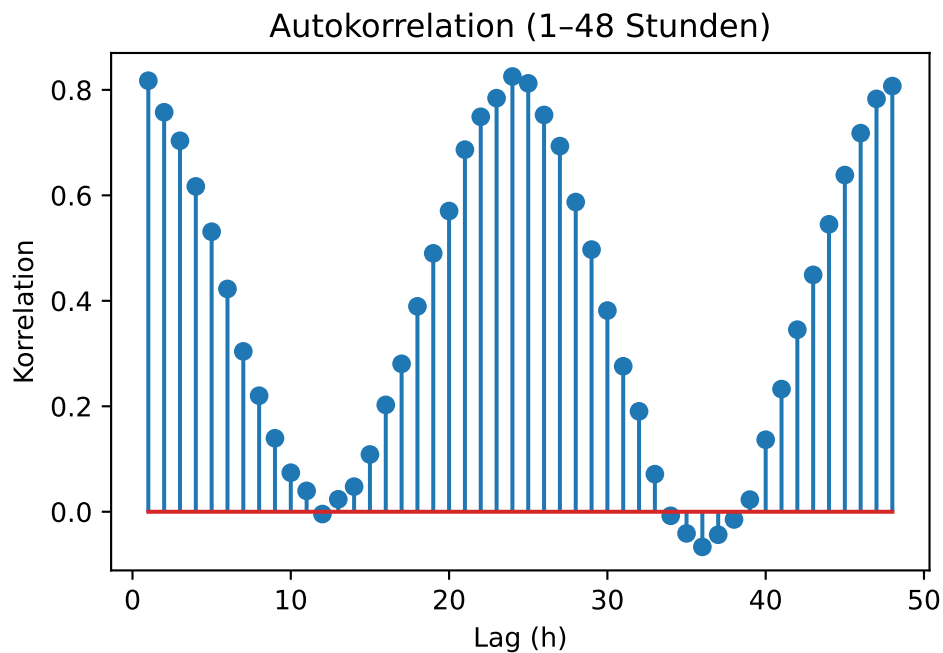
12.5 6.3 Autokorrelation berechnen

Autokorrelation misst die Korrelation zwischen $y(t)$ und $y(t-k)$.

12.6 Pandas

```
lags = range(1, 49)
ac = [s.corr(s.shift(k)) for k in lags]

plt.stem(list(lags), ac)
plt.title("Autokorrelation (1-48 Stunden)")
plt.xlabel("Lag (h)")
plt.ylabel("Korrelation")
plt.show()
```



12.7 NumPy (Prinzip)

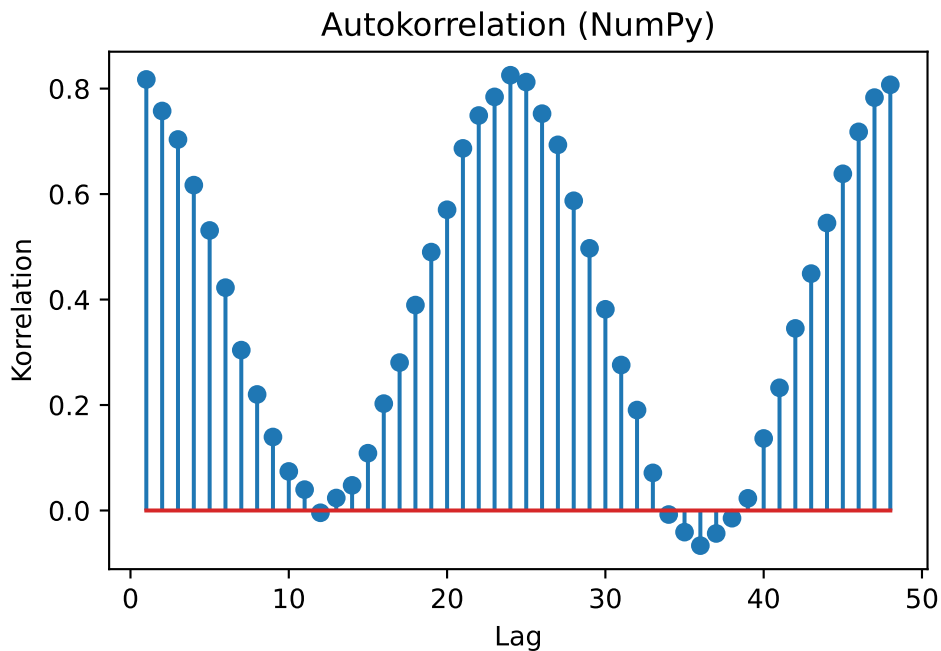
```
import numpy as np

y = s.to_numpy()
y = y - np.nanmean(y)

def autocorr(y, k):
    return np.corrcoef(y[:-k], y[k:])[0,1]
```

```
lags = range(1, 49)
ac_np = [autocorr(y, k) for k in lags]

plt.stem(list(lags), ac_np)
plt.title("Autokorrelation (NumPy)")
plt.xlabel("Lag")
plt.ylabel("Korrelation")
plt.show()
```



12.8 6.4 Interpretation der Autokorrelation

Typische Muster:

- Langsam abfallende Kurve → starker Trend
- Periodische Peaks (z. B. bei 24, 48, 72) → Saison
- Schneller Abfall gegen 0 → schwache Abhängigkeit

 Achtung

Trend kann Autokorrelation künstlich erhöhen. Gegebenenfalls zuerst detrenden oder differenzieren.

12.9 6.5 Mini-Aufgaben

1. Entfernen Sie den Trend (z. B. durch Rolling Mean).
 - Wie verändert sich die Autokorrelation?
2. Erzeugen Sie eine Zeitreihe ohne Saison.
 - Wie sieht die Autokorrelationsfunktion aus?
3. Erzeugen Sie reine Zufallsdaten.
 - Was erwarten Sie?

13 7 Anomalien

14 7 Anomalien

Anomalien sind Beobachtungen, die deutlich vom erwarteten Verhalten abweichen.

In Zeitreihen bedeutet das:

- starke Abweichung vom Trend
- starke Abweichung von saisonalem Muster
- plötzliche Strukturbrüche

Wichtig ist: Anomalien sind **kontextabhängig**.

i Merke

Eine Anomalie ist keine rein statistische Eigenschaft. Sie ist eine Abweichung vom erwarteten Prozessverhalten.

14.1 7.1 Beispielsignal mit Ausreißern

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

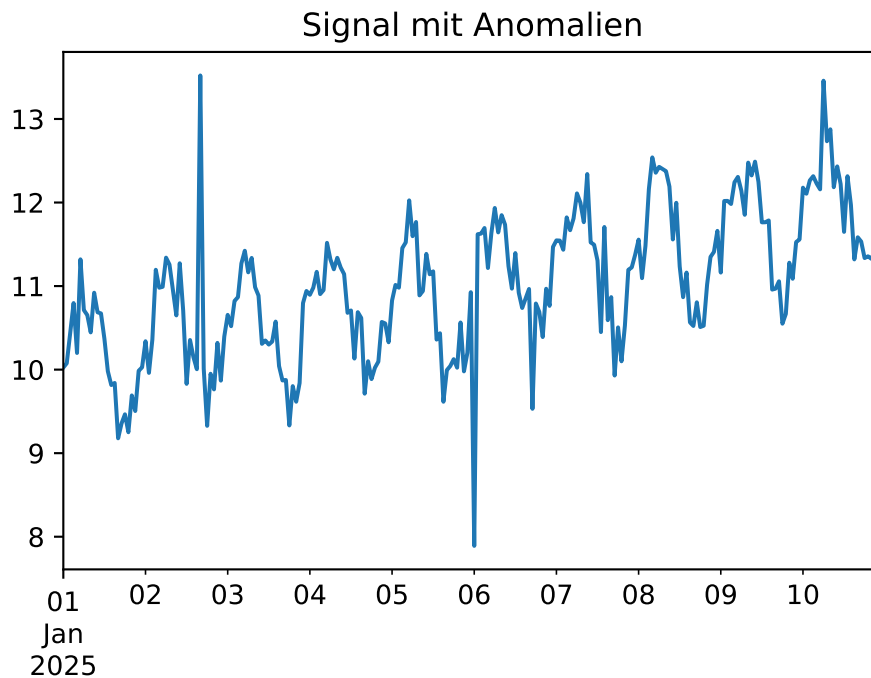
rng = np.random.default_rng(99)

t = pd.date_range("2025-01-01", periods=24*10, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.8 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.3 * rng.normal(size=len(t))
```

```
s = pd.Series(10 + trend + season + noise, index=t, name="signal")

# künstliche Anomalien
s.iloc[40] += 4
s.iloc[120] -= 3

s.plot(title="Signal mit Anomalien")
plt.show()
```



14.2 7.2 Globale Z-Score Methode

Wir standardisieren die gesamte Zeitreihe.

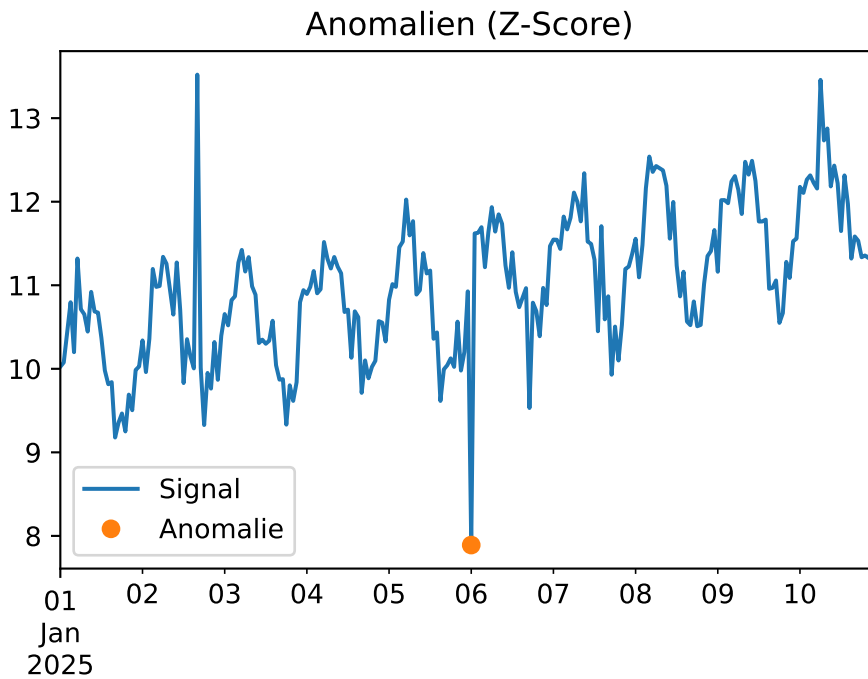
14.3 Pandas

```

z = (s - s.mean()) / s.std()
anomalies = s[abs(z) > 3]

ax = s.plot(label="Signal")
anomalies.plot(ax=ax, linestyle="None", marker="o", label="Anomalie")
plt.legend()
plt.title("Anomalien (Z-Score)")
plt.show()

```



14.4 NumPy

```

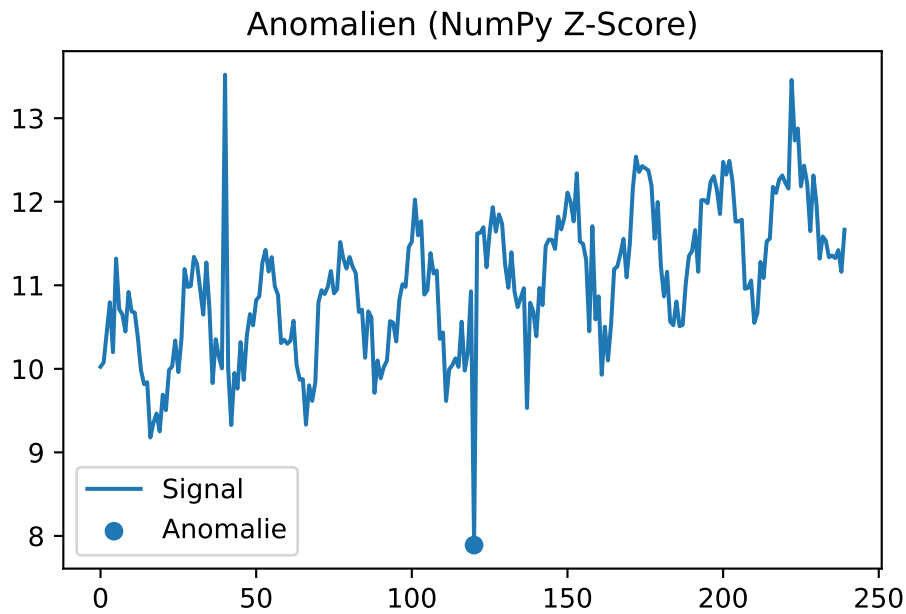
y = s.to_numpy()

z_np = (y - np.mean(y)) / np.std(y)
anomalies_idx = np.where(np.abs(z_np) > 3)[0]

plt.plot(y, label="Signal")
plt.scatter(anomalies_idx, y[anomalies_idx], label="Anomalie")
plt.legend()

```

```
plt.title("Anomalien (NumPy Z-Score)")
plt.show()
```



⚠ Achtung

Z-Score funktioniert schlecht bei starkem Trend oder Saison, weil globale Mittelwerte verzerren.

14.5 7.3 Lokale Anomalie (Baseline + Residuum)

Statt globaler Standardisierung definieren wir eine lokale Baseline.

```
baseline = s.ewm(span=24, adjust=False).mean()
residual = s - baseline

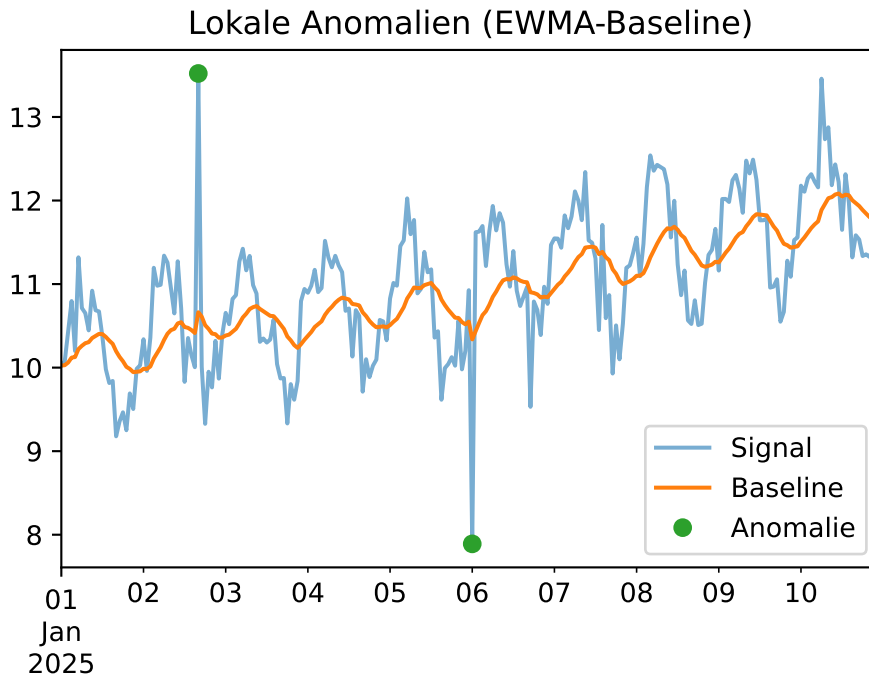
score = (residual - residual.mean()) / residual.std()
```

```

anomalies_local = s[abs(score) > 3]

ax = s.plot(alpha=0.6, label="Signal")
baseline.plot(ax=ax, label="Baseline")
anomalies_local.plot(ax=ax, linestyle="None", marker="o", label="Anomalie")
plt.legend()
plt.title("Lokale Anomalien (EWMA-Baseline)")
plt.show()

```



💡 Interpretation

Lokale Verfahren sind robuster, wenn Trend oder Saison vorhanden sind.

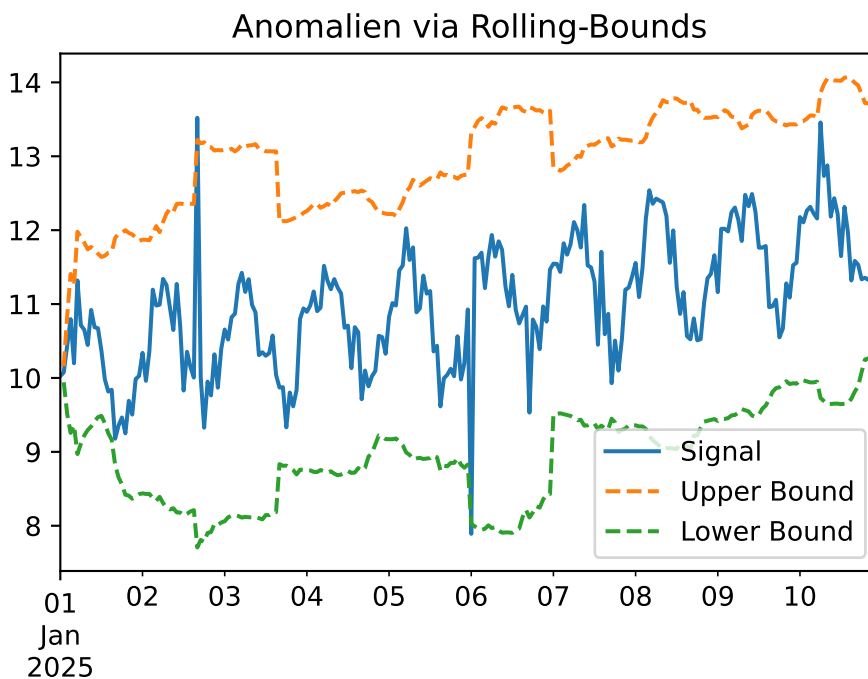
14.6 7.4 Rolling-Standardabweichung

Anomalien können auch als starke Abweichung relativ zur lokalen Varianz definiert werden.

```
rolling_std = s.rolling("24h").std()
rolling_mean = s.rolling("24h").mean()

upper = rolling_mean + 3 * rolling_std
lower = rolling_mean - 3 * rolling_std

ax = s.plot(label="Signal")
upper.plot(ax=ax, linestyle="--", label="Upper Bound")
lower.plot(ax=ax, linestyle="--", label="Lower Bound")
plt.legend()
plt.title("Anomalien via Rolling-Bounds")
plt.show()
```



14.7 7.5 Strukturbruch vs Ausreißer

Ein einzelner Peak → Ausreißer

Dauerhafte Niveauänderung → Strukturbruch

Beide erfordern unterschiedliche Interpretation.

i Methodischer Hinweis

Bevor Sie Anomalien markieren:

1. Trend prüfen
2. Saison prüfen
3. Baseline definieren
4. Schwellenwert begründen

14.8 7.6 Mini-Aufgaben

1. Entfernen Sie den Trend vor Anwendung des Z-Scores.
 - Wie verändert sich das Ergebnis?
2. Verringern Sie den Schwellenwert von 3 auf 2.
 - Was passiert?
3. Simulieren Sie einen dauerhaften Niveau-Sprung.
 - Wird dieser als Anomalie erkannt?

15 8 Forecasting-Baselines & Bewertung

16 8 Forecasting-Baselines & Bewertung

Bevor komplexe Modelle eingesetzt werden, sollten immer einfache **Baseline-Modelle** definiert werden.

Baselines beantworten die Frage:

Ist mein Modell wirklich besser als eine einfache, nachvollziehbare Strategie?

i Merke

Ein komplexes Modell ist nur dann sinnvoll, wenn es robuste Baselines klar übertrifft.

16.1 8.1 Beispielzeitreihe

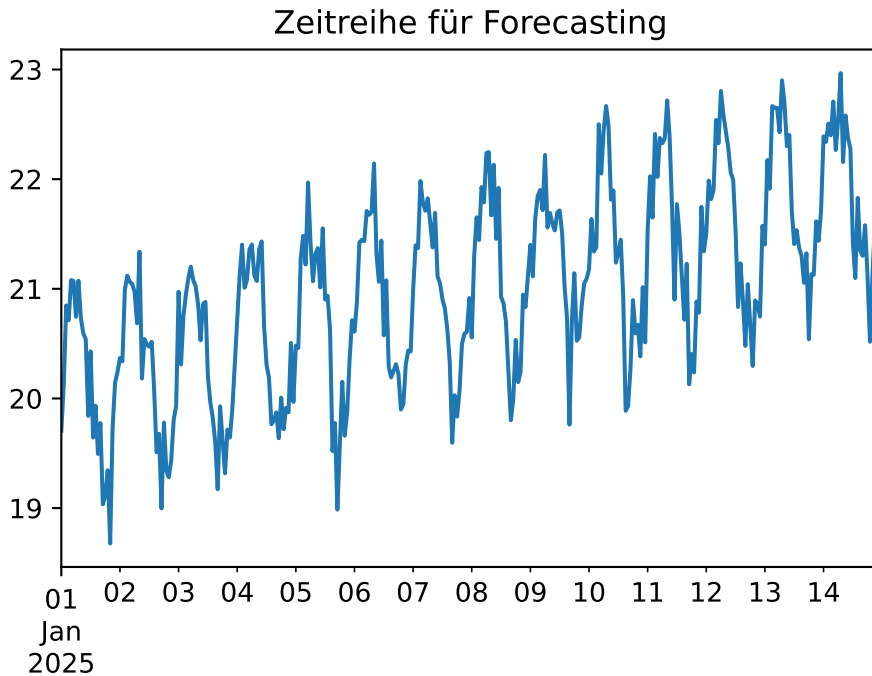
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(123)

t = pd.date_range("2025-01-01", periods=24*14, freq="h")
trend = np.linspace(0, 2, len(t))
season = 0.9 * np.sin(2*np.pi*(t.hour)/24)
noise = 0.3 * rng.normal(size=len(t))

s = pd.Series(20 + trend + season + noise, index=t, name="signal")

s.plot(title="Zeitreihe für Forecasting")
plt.show()
```



16.2 8.2 Train-Test-Split

Wir verwenden 80% der Daten für Training und 20% für Test.

```
split = int(len(s) * 0.8)

train = s.iloc[:split]
test = s.iloc[split:]

train.index.min(), train.index.max(), test.index.min(), test.index.max()
```

```
(Timestamp('2025-01-01 00:00:00'),
 Timestamp('2025-01-12 03:00:00'),
 Timestamp('2025-01-12 04:00:00'),
 Timestamp('2025-01-14 23:00:00'))
```

16.3 8.3 Naive Forecast

Nächster Wert = letzter beobachteter Wert.

16.4 Pandas

```
naive = test.copy()
naive[:] = train.iloc[-1]
```

16.5 NumPy

```
y = s.to_numpy()
train_np = y[:split]
test_np = y[split:]

naive_np = np.full_like(test_np, train_np[-1])
```

16.6 8.4 Seasonal Naive (24 Stunden)

Nächster Wert = Wert von vor 24 Stunden.

16.7 Pandas

```
seasonal_naive = s.shift(24).iloc[split:]
```

16.8 NumPy

```
seasonal_naive_np = y[split-24:-24]
```

16.9 8.5 Moving Average Baseline

Forecast = Mittelwert der letzten 24 Stunden.

16.10 Pandas

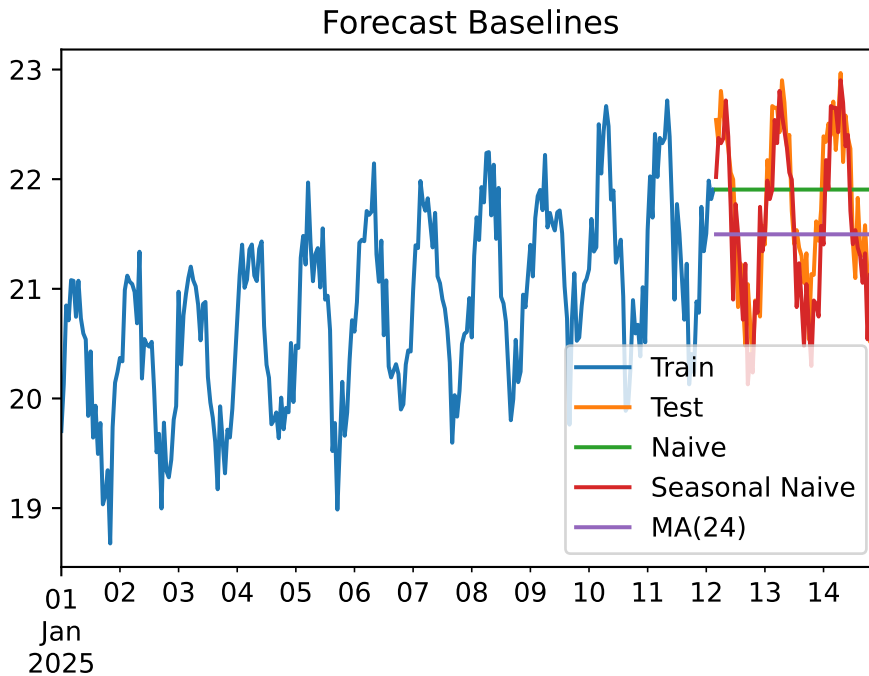
```
ma_value = train.iloc[-24:].mean()
ma_forecast = test.copy()
ma_forecast[:] = ma_value
```

16.11 NumPy

```
ma_value_np = np.mean(train_np[-24:])
ma_forecast_np = np.full_like(test_np, ma_value_np)
```

16.12 8.6 Vergleich visualisieren

```
plt.figure()
train.plot(label="Train")
test.plot(label="Test")
naive.plot(label="Naive")
seasonal_naive.plot(label="Seasonal Naive")
ma_forecast.plot(label="MA(24)")
plt.legend()
plt.title("Forecast Baselines")
plt.show()
```



16.13 8.7 Fehlermaße

Wir verwenden den Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum |y_{true} - y_{pred}|$$

16.14 Pandas

```
def mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

print("MAE Naive:", mae(test, naive))
print("MAE Seasonal Naive:", mae(test, seasonal_naive))
print("MAE MA(24):", mae(test, ma_forecast))
```

MAE Naive: 0.6298837368696829
MAE Seasonal Naive: 0.3320412574311427
MAE MA(24): 0.6228787570177815

16.15 NumPy

```
def mae_np(y_true, y_pred):  
    return np.mean(np.abs(y_true - y_pred))  
  
print("MAE Naive:", mae_np(test_np, naive_np))  
print("MAE Seasonal:", mae_np(test_np, seasonal_naive_np))  
print("MAE MA(24):", mae_np(test_np, ma_forecast_np))
```

MAE Naive: 0.6298837368696829
MAE Seasonal: 0.3320412574311427
MAE MA(24): 0.6228787570177815

16.16 8.8 Interpretation

Typische Beobachtungen:

- Bei starker Saison gewinnt häufig Seasonal Naive.
 - Bei schwachem Muster ist Naive oft überraschend stark.
 - Moving Average glättet, reagiert aber träge auf Trendänderungen.
-

Methodischer Hinweis

Baseline-Vergleich ist keine Formalität, sondern eine Qualitätskontrolle.

16.17 8.9 Mini-Aufgaben

1. Erhöhen Sie die Stärke der Saison.
 - Welche Baseline gewinnt?
2. Entfernen Sie die Saison vollständig.
 - Welche Baseline wird besser?
3. Fügen Sie einen starken Trend hinzu.
 - Wie reagieren die Modelle?

17 9 Lernzielkontrolle

18 9 Lernzielkontrolle

Nutzen Sie diese Checkliste zur Selbstüberprüfung.

18.1 9.1 Fachliche Kompetenzen

Ich kann ...

- ☐ eine Zeitreihe methodisch einordnen (Messprozess, Abtastrate).
 - ☐ eine Zeitreihe validieren (Sortierung, Duplikate, Missing Values).
 - ☐ Resampling begründet einsetzen (Aggregation vs. Interpolation).
 - ☐ Rolling Mean und EWMA unterscheiden und passend wählen.
 - ☐ Trend und Saison konzeptionell trennen.
 - ☐ Lag-Features erzeugen und interpretieren.
 - ☐ Autokorrelation interpretieren.
 - ☐ Anomalien global und lokal definieren.
 - ☐ Baseline-Forecasts implementieren.
 - ☐ Forecast-Modelle mit MAE bewerten.
-

18.2 9.2 Verständnisfragen

1. Warum ist die Abtastrate für die Interpretation einer Saison entscheidend?
 2. Wann ist Interpolation problematisch?
 3. Warum kann ein Trend die Autokorrelation künstlich erhöhen?
 4. Warum sind Baselines methodisch wichtig?
 5. Wodurch unterscheidet sich ein Strukturbruch von einem Ausreißer?
-

 Hinweis

Wenn mehrere Punkte unsicher sind, wiederholen Sie gezielt die entsprechenden Kapitel und bearbeiten Sie die Übungsaufgaben.

19 10 Übung: Zeitreihen-Workflow

20 10 Übung: Zeitreihen-Workflow

Diese Übung kombiniert mehrere Methoden aus dem Baustein.

20.1 Aufgabe 1: Eigene Zeitreihe erzeugen

Erzeugen Sie eine Zeitreihe mit:

- 10 Tagen, stündlicher Abtastrate
- Trend
- Tages-Saison
- Rauschen
- mindestens 3 Anomalien

Visualisieren Sie die Zeitreihe.

20.2 Aufgabe 2: Validierung

Bauen Sie absichtlich Fehler ein:

- Duplikate
- Missing Values
- Unsortierte Zeitpunkte

Implementieren Sie eine saubere Validierung.

20.3 Aufgabe 3: Resampling

- Aggregieren Sie auf Tagesmittel.
 - Vergleichen Sie Original und Aggregation.
 - Interpretieren Sie Informationsverlust.
-

20.4 Aufgabe 4: Glättung

- Testen Sie Rolling-Fenster von 6H, 24H, 72H.
 - Welche Struktur wird jeweils sichtbar?
-

20.5 Aufgabe 5: Anomalien

- Implementieren Sie eine EWMA-Baseline.
 - Detektieren Sie Anomalien.
 - Variieren Sie den Schwellenwert.
-

20.6 Aufgabe 6: Forecast-Baselines

- Train-Test-Split (80/20)
- Naive
- Seasonal Naive (24H)
- MA(24)

Vergleichen Sie die MAE-Werte und interpretieren Sie das Ergebnis.

i Abgabeformat (optional)

Plots + kurze Interpretation (Stichpunkte).

21 11 Klausurfragen

22 11 Klausurfragen

22.1 Verständnisfragen

1. Definieren Sie eine Zeitreihe.
 2. Was ist der Unterschied zwischen Aggregation und Interpolation?
 3. Erklären Sie Rolling Mean und EWMA konzeptionell.
 4. Was ist ein Lag-Feature?
 5. Was misst die Autokorrelation?
 6. Warum kann Trend die Autokorrelation beeinflussen?
 7. Erklären Sie den Unterschied zwischen globaler und lokaler Anomalie-Detektion.
 8. Was ist eine Forecast-Baseline?
 9. Warum sind Baselines methodisch notwendig?
 10. Was misst der MAE?
-

22.2 Transferfragen

1. Sie erhalten Maschinendaten mit starker Wochenperiodik.
 - Welche Schritte würden Sie durchführen?
 2. Eine Zeitreihe zeigt plötzlich dauerhaft höhere Werte.
 - Ausreißer oder Strukturbruch? Begründen Sie.
 3. Zwei Zeitreihen sollen verglichen werden, eine minütlich, eine stündlich gemessen.
 - Welche Schritte sind notwendig?
-

 Hinweis zur Prüfungsvorbereitung

Begründen Sie Ihre Antworten stets methodisch. Reine Definitionen reichen meist nicht aus.